

编译器设计专题实验报告

实验四：SLR(1) 分析表生成

张凌曜 2234214014 计算机 2306

1 实验目的

在实验三 LR(0) 规范族的基础上，算 FIRST 集和 FOLLOW 集，生成 SLR(1) 分析表 (ACTION 和 GOTO)，利用 FOLLOW 集解决 LR(0) 中移进-归约冲突。

2 实验原理

LR(0) 遇到归约项目就无脑归约，SLR(1) 改进了一点：只有当前输入符号在 FOLLOW(A) 里才归约，这样能消掉不少冲突。

FIRST 集就是从某个符号串能推导出来的第一个终结符。FOLLOW 集是在某些句型中能紧跟在非终结符后面的终结符。FOLLOW 集的计算要迭代，直到集合不再变化。

3 实验过程

3.1 FIRST/FOLLOW 集计算

两个集合都用迭代法，反复扫所有产生式直到不动了为止。FOLLOW 集的规则有三条：起始符号的 FOLLOW 加 \$； $A \rightarrow \alpha B \beta$ 时把 FIRST(β) 里非 ϵ 的元素加到 FOLLOW(B)；如果 β 能推出 ϵ 就把 FOLLOW(A) 也加到 FOLLOW(B)。

3.2 分析表构建

对每个状态 I_i 里的每个项目：点后面是终结符 a 就填移进 s_j ；归约项目 $A \rightarrow \alpha \cdot$ 就在 FOLLOW(A) 对应的列填归约 r_k ； $S' \rightarrow S \cdot$ 在 \$ 列填 acc。

4 测试结果

4.1 四则运算文法

FOLLOW 集结果:

$$\begin{aligned} \text{FOLLOW}(E) &= \{\$,), +\} & \text{FOLLOW}(E') &= \{\$ \} \\ \text{FOLLOW}(T) &= \{\$,), *, +\} & \text{FOLLOW}(F) &= \{\$,), *, +\} \end{aligned}$$

ACTION 表 (部分):

状态	(	)	*	+	id	\$
I_0	s1				s5	
I_2				s7		acc
I_3			r4	r4	r4	r4
I_4			r2	s8	r2	r2
I_5			r6	r6	r6	r6

I_2 里 $E' \rightarrow E \cdot$ 是归约, $E \rightarrow E \cdot + T$ 是移进。LR(0) 下这是冲突, 但 SLR(1) 里归约只在 $\text{FOLLOW}(E') = \{\$ \}$ 时执行, 移进在 $+$ 时执行, 两边不重叠, 冲突解决了。

```

===== LR(0) 分析表 =====
(1) E -> E
(2) E -> T
(3) T -> T
(4) T -> F
(5) F -> ( E )
(6) F -> id

===== FOLLOW 集 =====
FOLLOW(E) = { $, ) }
FOLLOW(T) = { $, ) }
FOLLOW(F) = { $, ) }
FOLLOW(id) = { $, ) }

===== LR(0) 项目集 =====
I0: E -> E
I1: E -> T
I2: T -> T
I3: T -> F
I4: F -> ( E
I5: F -> id

===== SLR(1) 分析表 =====
--- ACTION 表 ---
状态 ( ) * + id $
I0 s1
I1 s1
I2
I3 r4 r4 r4
I4 r2 s8 r2
I5 r6 r6 r6

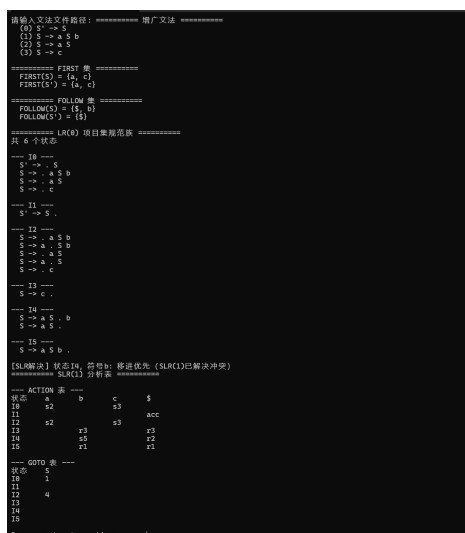
--- GOTO 表 ---
状态 E F T
I0 2 3 4
I1 6 3 4
I2
I3
I4
I5
I6
I7
I8
I9
I10
I11
I12
I13
I14
I15
I16
I17
I18
I19
I20
I21
I22
I23
I24
I25
I26
I27
I28
I29
I30
I31
I32
I33
I34
I35
I36
I37
I38
I39
I40
I41
I42
I43
I44
I45
I46
I47
I48
I49
I50
I51
I52
I53
I54
I55
I56
I57
I58
I59
I60
I61
I62
I63
I64
I65
I66
I67
I68
I69
I70
I71
I72
I73
I74
I75
I76
I77
I78
I79
I80
I81
I82
I83
I84
I85
I86
I87
I88
I89
I90
I91
I92
I93
I94
I95
I96
I97
I98
I99
I100
I101
I102
I103
I104
I105
I106
I107
I108
I109
I110
I111
I112
I113
I114
I115
I116
I117
I118
I119
I120
I121
I122
I123
I124
I125
I126
I127
I128
I129
I130
I131
I132
I133
I134
I135
I136
I137
I138
I139
I140
I141
I142
I143
I144
I145
I146
I147
I148
I149
I150
I151
I152
I153
I154
I155
I156
I157
I158
I159
I160
I161
I162
I163
I164
I165
I166
I167
I168
I169
I170
I171
I172
I173
I174
I175
I176
I177
I178
I179
I180
I181
I182
I183
I184
I185
I186
I187
I188
I189
I190
I191
I192
I193
I194
I195
I196
I197
I198
I199
I200
I201
I202
I203
I204
I205
I206
I207
I208
I209
I210
I211
I212
I213
I214
I215
I216
I217
I218
I219
I220
I221
I222
I223
I224
I225
I226
I227
I228
I229
I230
I231
I232
I233
I234
I235
I236
I237
I238
I239
I240
I241
I242
I243
I244
I245
I246
I247
I248
I249
I250
I251
I252
I253
I254
I255
I256
I257
I258
I259
I260
I261
I262
I263
I264
I265
I266
I267
I268
I269
I270
I271
I272
I273
I274
I275
I276
I277
I278
I279
I280
I281
I282
I283
I284
I285
I286
I287
I288
I289
I290
I291
I292
I293
I294
I295
I296
I297
I298
I299
I300
I301
I302
I303
I304
I305
I306
I307
I308
I309
I310
I311
I312
I313
I314
I315
I316
I317
I318
I319
I320
I321
I322
I323
I324
I325
I326
I327
I328
I329
I330
I331
I332
I333
I334
I335
I336
I337
I338
I339
I340
I341
I342
I343
I344
I345
I346
I347
I348
I349
I350
I351
I352
I353
I354
I355
I356
I357
I358
I359
I360
I361
I362
I363
I364
I365
I366
I367
I368
I369
I370
I371
I372
I373
I374
I375
I376
I377
I378
I379
I380
I381
I382
I383
I384
I385
I386
I387
I388
I389
I390
I391
I392
I393
I394
I395
I396
I397
I398
I399
I400
I401
I402
I403
I404
I405
I406
I407
I408
I409
I410
I411
I412
I413
I414
I415
I416
I417
I418
I419
I420
I421
I422
I423
I424
I425
I426
I427
I428
I429
I430
I431
I432
I433
I434
I435
I436
I437
I438
I439
I440
I441
I442
I443
I444
I445
I446
I447
I448
I449
I450
I451
I452
I453
I454
I455
I456
I457
I458
I459
I460
I461
I462
I463
I464
I465
I466
I467
I468
I469
I470
I471
I472
I473
I474
I475
I476
I477
I478
I479
I480
I481
I482
I483
I484
I485
I486
I487
I488
I489
I490
I491
I492
I493
I494
I495
I496
I497
I498
I499
I500
I501
I502
I503
I504
I505
I506
I507
I508
I509
I510
I511
I512
I513
I514
I515
I516
I517
I518
I519
I520
I521
I522
I523
I524
I525
I526
I527
I528
I529
I530
I531
I532
I533
I534
I535
I536
I537
I538
I539
I540
I541
I542
I543
I544
I545
I546
I547
I548
I549
I550
I551
I552
I553
I554
I555
I556
I557
I558
I559
I560
I561
I562
I563
I564
I565
I566
I567
I568
I569
I570
I571
I572
I573
I574
I575
I576
I577
I578
I579
I580
I581
I582
I583
I584
I585
I586
I587
I588
I589
I590
I591
I592
I593
I594
I595
I596
I597
I598
I599
I600
I601
I602
I603
I604
I605
I606
I607
I608
I609
I610
I611
I612
I613
I614
I615
I616
I617
I618
I619
I620
I621
I622
I623
I624
I625
I626
I627
I628
I629
I630
I631
I632
I633
I634
I635
I636
I637
I638
I639
I640
I641
I642
I643
I644
I645
I646
I647
I648
I649
I650
I651
I652
I653
I654
I655
I656
I657
I658
I659
I660
I661
I662
I663
I664
I665
I666
I667
I668
I669
I670
I671
I672
I673
I674
I675
I676
I677
I678
I679
I680
I681
I682
I683
I684
I685
I686
I687
I688
I689
I690
I691
I692
I693
I694
I695
I696
I697
I698
I699
I700
I701
I702
I703
I704
I705
I706
I707
I708
I709
I710
I711
I712
I713
I714
I715
I716
I717
I718
I719
I720
I721
I722
I723
I724
I725
I726
I727
I728
I729
I730
I731
I732
I733
I734
I735
I736
I737
I738
I739
I740
I741
I742
I743
I744
I745
I746
I747
I748
I749
I750
I751
I752
I753
I754
I755
I756
I757
I758
I759
I760
I761
I762
I763
I764
I765
I766
I767
I768
I769
I770
I771
I772
I773
I774
I775
I776
I777
I778
I779
I780
I781
I782
I783
I784
I785
I786
I787
I788
I789
I790
I791
I792
I793
I794
I795
I796
I797
I798
I799
I800
I801
I802
I803
I804
I805
I806
I807
I808
I809
I810
I811
I812
I813
I814
I815
I816
I817
I818
I819
I820
I821
I822
I823
I824
I825
I826
I827
I828
I829
I830
I831
I832
I833
I834
I835
I836
I837
I838
I839
I840
I841
I842
I843
I844
I845
I846
I847
I848
I849
I850
I851
I852
I853
I854
I855
I856
I857
I858
I859
I860
I861
I862
I863
I864
I865
I866
I867
I868
I869
I870
I871
I872
I873
I874
I875
I876
I877
I878
I879
I880
I881
I882
I883
I884
I885
I886
I887
I888
I889
I890
I891
I892
I893
I894
I895
I896
I897
I898
I899
I900
I901
I902
I903
I904
I905
I906
I907
I908
I909
I910
I911
I912
I913
I914
I915
I916
I917
I918
I919
I920
I921
I922
I923
I924
I925
I926
I927
I928
I929
I930
I931
I932
I933
I934
I935
I936
I937
I938
I939
I940
I941
I942
I943
I944
I945
I946
I947
I948
I949
I950
I951
I952
I953
I954
I955
I956
I957
I958
I959
I960
I961
I962
I963
I964
I965
I966
I967
I968
I969
I970
I971
I972
I973
I974
I975
I976
I977
I978
I979
I980
I981
I982
I983
I984
I985
I986
I987
I988
I989
I990
I991
I992
I993
I994
I995
I996
I997
I998
I999

```

4.2 嵌套文法

$S \rightarrow aSb|aS|c$

I_4 里有 $S \rightarrow aS \cdot b$ (移进) 和 $S \rightarrow aS \cdot$ (归约)。SLR(1) 看 $\text{FOLLOW}(S) = \{\$, b\}$, b 在 FOLLOW 里所以移进和归约对 b 都有定义, 冲突没法完全消掉。程序里对 b 优先移进, 并输出了冲突提示。



5 选做：flex/bison 自动化工具

用 flex 做词法分析、bison 做语法分析，实现了一个四则运算计算器。

5.1 flex 词法定义 (calc.l)

flex 用正则表达式定义 Token 规则，自动生成了扫描器 lex.yy.c:

```

1  [ \t\r\n]+      { /* skip whitespace */ }
2  [0-9]+          { yylval = atoi(yytext); return NUM; }
3  "+"            { return PLUS; }
4  "*"            { return TIMES; }
5  "("            { return LPAREN; }
6  ")"            { return RPAREN; }

```

5.2 bison 语法定义 (calc.y)

bison 用 BNF 定义文法和语义动作，自动生成了 LALR(1) 分析器 calc.tab.c:

```

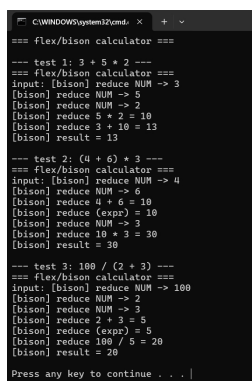
1  %left PLUS MINUS
2  %left TIMES DIVIDE
3
4  expr:
5      NUM                { $$ = $1; }
6      | expr PLUS expr   { $$ = $1 + $3; }
7      | expr TIMES expr  { $$ = $1 * $3; }
8      | LPAREN expr RPAREN { $$ = $2; }
9      ;

```

%left 声明自动解决了运算符优先级和结合性冲突，不需要手动构造分析表。

5.3 测试结果

输入 $3 + 5 * 2$, bison 按优先级先归约 $5 * 2 = 10$, 再归约 $3 + 10 = 13$:



```
C:\WINDOWS\system32\cmd.exe
flex/bison calculator
== test 1: 3 + 5 * 2 ==
flex/bison calculator
input: [bison] reduce NUM -> 3
[bison] reduce NUM -> 5
[bison] reduce NUM -> 2
[bison] reduce 5 * 2 = 10
[bison] reduce 3 + 10 = 13
[bison] result = 13

== test 2: (4 + 6) * 3 ==
flex/bison calculator
input: [bison] reduce NUM -> 4
[bison] reduce NUM -> 6
[bison] reduce 4 + 6 = 10
[bison] reduce (expr) = 10
[bison] reduce NUM -> 3
[bison] reduce 10 * 3 = 30
[bison] result = 30

== test 3: 100 / (2 + 3) ==
flex/bison calculator
input: [bison] reduce NUM -> 100
[bison] reduce NUM -> 2
[bison] reduce NUM -> 3
[bison] reduce 2 + 3 = 5
[bison] reduce (expr) = 5
[bison] reduce 100 / 5 = 20
[bison] result = 20

Press any key to continue . . .
```

对比手写的 SLR(1) 程序: flex/bison 把词法分析和语法分析的代码生成自动化了,只需要写 .l 和 .y 两个定义文件,就能得到完整的分析器。这就是工业界用 yacc/bison 构建编译器的原因。

6 实验总结

SLR(1) 本质上就是“归约的时候往前看一眼”,用 FOLLOW 集判断该不该归约。大部分常见冲突都能解决,但像嵌套文法那种就不行了,得上 LALR(1) 或者 LR(1)。

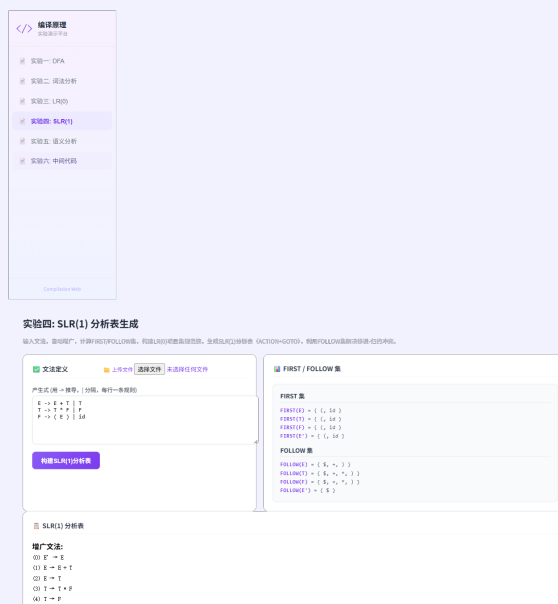
FOLLOW 集的迭代计算要注意收敛条件,必须集合完全不变才停,不然会死循环。

选做部分用 flex/bison 体验了一下自动化工具,发现它们和手写 SLR(1) 的核心思路是一样的——都是基于 LALR(1) 分析,只不过 flex/bison 自动帮你做了 FIRST/FOLLOW 集计算和冲突解决。这让人更能体会到理论到工程的桥梁作用。

Web 前端展示

本实验的 SLR(1) 分析表生成功能已实现为 Web 前端页面。

主要功能：文法输入、FIRST 集和 FOLLOW 集计算过程展示、ACTION 表与 GOTO 表生成（移进/归约/acc/goto 用不同颜色标注）、冲突自动检测与提示。



前端支持四则运算文法、嵌套文法等多种测试用例, 分析表单元格按类型着色, 便于直观检查冲突解决结果。

A 附录：代码仓库

本实验的全部源码、报告及 Web 演示平台已托管至 GitHub:

<https://github.com/yaossss123/compilation-web>